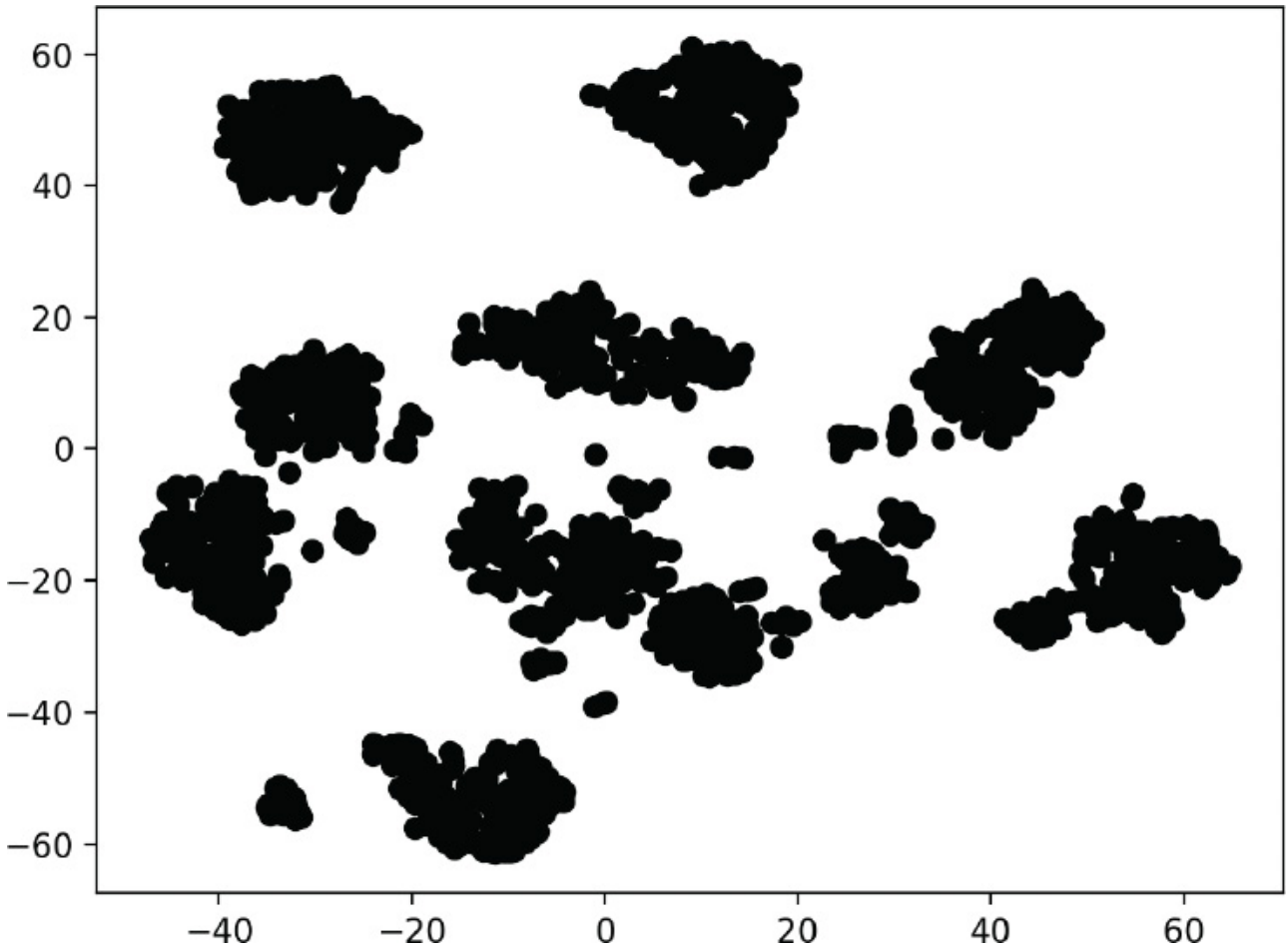


Visualizing the Reduced Data with Different Colors for Each Digit

Though the preceding diagram shows clusters, we do not know whether all the items in each cluster represent the same digit. If they do not, then the clusters are not helpful. Let's use the known `target`s in the Digits dataset to color all the dots so we can see whether these clusters indeed represent specific digits:

lick here to view code image

```
In [9]: dots = plt.scatter(reduced_data[:, 0], reduced_data[:, 1],
   ...:     c=digits.target, cmap=plt.cm.get_cmap('nipy_spectral_r', 10))
   ...:
   ...:
```

In this case, `scatter`'s keyword argument `c=digits.target` specifies that the `target` values determine the dot colors. We also added the keyword argument

lick here to view code image

```
cmap=plt.cm.get_cmap('nipy_spectral_r', 10)
```

C

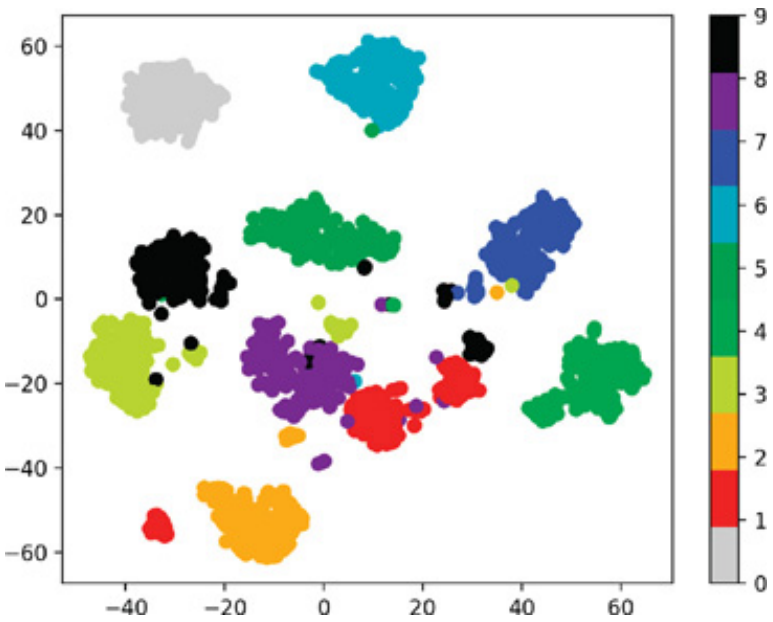
which specifies a color map to use when coloring the dots. In this case, we know we're coloring 10 digits, so we use `get_cmap` method of Matplotlib's `cm` object (from module `matplotlib.pyplot`) to load a color map (`'nipy_spectral_r'`) and select `10` distinct colors from the color map.

The following statement adds a color bar key to the right of the diagram so you can see which digit each color represents:

lick here to view code image

```
In [10]: colorbar = plt.colorbar(dots)
```

Voila! We see 10 clusters corresponding to the digits 0–9. Again, there are a few smaller groups of dots standing alone. Based on this, we might decide that a supervised-learning approach like k-nearest neighbors would work well with this data. As an experiment, you might want to investigate Matplotlib's **`Axes3D`**, which provides *x*-, *y*- and *z*-axes for plotting in three-dimensional graphs.



14.7 CASE STUDY: UNSUPERVISED MACHINE LEARNING, PART 2—K-MEANS CLUSTERING

In this section, we introduce perhaps the simplest unsupervised machine learning algorithms—**k-means clustering**. This algorithm analyzes *unlabeled* samples and attempts to place them in clusters that appear to be related. The *k* in "k-means" represents the number of clusters you'd like to see imposed on your data.

The algorithm organizes samples into the number of clusters you specify in advance, using distance calculations similar to the k-nearest neighbors clustering algorithm.

C

ach cluster of samples is grouped around a **centroid**—the cluster's center point. Initially, the algorithm chooses *k* centroids at random from the dataset's samples. Then the remaining samples are placed in the cluster whose centroid is the closest. The centroids are iteratively recalculated and the samples re-assigned to clusters until, for all clusters, the distances from a given centroid to the samples in its cluster are minimized. The algorithm's results are:

- a one-dimensional array of labels indicating the cluster to which each sample belongs and
- a two-dimensional array of centroids representing the center of each cluster.

Iris Dataset

We'll work with the popular **Iris dataset** [2] bundled with scikit-learn, which is commonly analyzed with both classification and clustering. Although this dataset is labeled, we'll ignore those labels here to demonstrate clustering. Then, we'll use the labels to determine how well the k-means algorithm clustered the samples.

[2]Fisher, R.A., The use of multiple measurements in taxonomic problems, Annual Eugenics, 7, Part II, 179-188 (1936); also in Contributions to Mathematical Statistics (John Wiley, NY, 1950).

The Iris dataset is referred to as a "toy dataset" because it has only 150 samples and four features. The dataset describes 50 samples for each of three *Iris* flower species —*Iris setosa*, *Iris versicolor* and *Iris virginica*. Photos of these are shown below. Each sample's features are the sepal length, sepal width, petal length and petal width, all measured in centimeters. The *sepals* are the larger outer parts of each flower that protect the smaller inside *petals* before the flower buds bloom.

E

Iris setosa:
https://commons.wikimedia.org/wiki/File:Wild_iris_KEFJ_(9025144383).jpg.

Credit: Courtesy of Nation Park services.

Iris versicolor: https://commons.wikimedia.org/wiki/Iris_versicolor#/media/

File:IrisVersicolor-FoxRoost-Newfoundland.jpg.

Credit: Courtesy of Jefficus,

https://commons.wikimedia.org/w/index.php?
title=User:Jefficus&action=edit&redlink=1

Iris virginica: https://commons.wikimedia.org/wiki/File:IMG_7911-Iris_virginica.jpg.

Credit: Christer T Johansson.

14.7.1 Loading the Iris Dataset

Launch IPython with `ipython --matplotlib`, then use the `sklearn.datasets` module's **`load_iris` function** to get a `Bunch` containing the dataset:

lick here to view code image

```
In [1]: from sklearn.datasets import load_iris

In [2]: iris = load_iris()
```

The `Bunch`'s `DESCR` attribute indicates that there are 150 samples (`Number of Instances`), each with four features (`Number of Attributes`). There are no missing values in this dataset. The dataset classifies the samples by labeling them with the integers 0, 1 and 2, representing *Iris setosa*, *Iris versicolor* and *Iris virginica*, respectively. We'll *ignore* the labels and let the k-means clustering algorithm try to

C

determine the samples' classes. We show some key `DESCR` information in bold.:

lick here to view code image

```
 n [3]: print(iris.DESCR)
.. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

    :Number of Instances: 150 (50 in each   of three classes)
    :Number of Attributes: 4   numeric, predictive attributes and the clas
    :Attribute Information:
        - sepal length in cm
        - sepal width in cm
        - petal length in cm
        - petal width in cm
        - class:
                - Iris-Setosa
                - Iris-Versicolour
                - Iris-Virginica

    :Summary Statistics:

    ============== ==== ==== ======= =====   ====================
                    Min  Max   Mean    SD   Class   Correlation
    ============== ==== ==== ======= =====   ====================
    sepal length:   4.3  7.9   5.84   0.83    0.7826
    sepal width:    2.0  4.4   3.05   0.43   -0.4194
    petal length:   1.0  6.9   3.76   1.76    0.9490    (high!)
    petal width:    0.1  2.5   1.20   0.76    0.9565    (high!)
    ============== ==== ==== ======= =====   ====================

    :Missing Attribute Values: None
    :Class Distribution: 33.3% for each of 3 classes.
    :Creator: R.A. Fisher
    :Donor: Michael Marshall   (MARSHALL%PLU@io.arc.nasa.gov)
    :Date: July, 1988

The famous Iris database, first used by Sir R.A. Fisher.   The dataset is
the UCI Machine Learning Repository, which has two wrong   data points.

This is perhaps the best known database to   be found in the pattern
recognition literature.  Fisher's   paper is a classic in the field and
is referenced frequently to this day.  (See Duda &   Hart, for example.)
refers to a type of iris plant.  One class is linearly   separable from th
other 2; the latter are NOT linearly separable from each   other.

.. topic:: References
```

C

```
      - Fisher, R.A. "The use of multiple measurements in   taxonomic
         problems"
        Annual Eugenics, 7, Part II, 179-188 (1936); also in   "Contributions
        to Mathematical Statistics" (John Wiley, NY,   1950).
      - Duda, R.O., & Hart, P.E. (1973) Pattern   Classification and Scene
        Analysis.
        (Q327.D83) John Wiley & Sons.  ISBN   0-471-22361-1.  See page 218.
      - Dasarathy, B.V. (1980) "Nosing Around the   Neighborhood: A New Syste
        Structure and Classification Rule for Recognition in   Partially
        Exposed Environments".  IEEE Transactions on   Pattern Analysis and
        Machine Intelligence, Vol. PAMI-2, No. 1, 67-71.
      - Gates, G.W. (1972) "The Reduced Nearest Neighbor   Rule".  IEEE
        Transactions on Information Theory, May 1972, 431-433.
      - See also: 1988 MLC Proceedings, 54-64.  Cheeseman et   al"s AUTOCLASS
        II conceptual clustering system finds 3 classes in the   data.
      - Many, many more ...
```

hecking the Numbers of Samples, Features and Targets

You can confirm the number of samples and features per sample via the `data` array's `shape`, and you can confirm the number of targets via the `target` array's `shape`:

lick here to view code image

```
In [4]: iris.data.shape
Out[4]: (150, 4)

In [5]: iris.target.shape
Out[5]: (150,)
```

The array `target_names` contains the names for the `target` array's numeric labels—`dtype='<U10'` indicates that the elements are strings with a maximum of 10 characters:

lick here to view code image

```
In [6]: iris.target_names
Out[6]: array(['setosa', 'versicolor', 'virginica'],   dtype='<U10')
```

The array `feature_names` contains a list of string names for each column in the `data` array:

lick here to view code image

C

```
In [7]: iris.feature_names
Out[7]:
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

14.7.2 Exploring the Iris Dataset: Descriptive Statistics with Pandas

Let's use a `DataFrame` to explore the Iris dataset. As we did in the California Housing case study, let's set the pandas options for formatting the column-based outputs:

lick here to view code image

```
In [8]: import pandas as pd

In [9]: pd.set_option('max_columns',   5)

In [10]: pd.set_option('display.width',   None)
```

Create a `DataFrame` containing the `data` array's contents, using the contents of the `feature_names` array as the column names:

lick here to view code image

```
In [11]: iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

Next, add a column containing each sample's species name. The list comprehension in the following snippet uses each value in the `target` array to look up the corresponding species name in the `target_names` array:

lick here to view code image

```
In [12]: iris_df['species'] =   [iris.target_names[i] for i in   iris.targ
```

et's use pandas' to look at a few samples. Once again notice that pandas displays a \ to the right of the column heads to indicate that there are more columns displayed below:

lick here to view code image

C

```
In [13]: iris_df.head()
Out[13]:
   sepal length (cm)  sepal width (cm)  petal length (cm)     \
0                5.1               3.5                1.4
1                4.9               3.0                1.4
2                4.7               3.2                1.3
3                4.6               3.1                1.5
4                5.0               3.6                1.4

   petal width (cm) species
0               0.2  setosa
1               0.2  setosa
2               0.2  setosa
3               0.2  setosa
4               0.2  setosa
```

Let's calculate some descriptive statistics for the numerical columns:

lick here to view code image

```
In [14]: pd.set_option('precision',   2)

In [15]: iris_df.describe()
Out[15]:
       sepal length (cm)  sepal width (cm)  petal length   (cm)  \
count             150.00            150.00               150.00
mean                5.84              3.06                 3.76
std                 0.83              0.44                 1.77
min                 4.30              2.00                 1.00
25%                 5.10              2.80                 1.60
50%                 5.80              3.00               4.35
75%                 6.40              3.30                 5.10
max                 7.90              4.40                 6.90

       petal width (cm)
count            150.00
mean               1.20
std                0.76
min                0.10
25%                0.30
50%                1.30
75%                1.80
max                2.50
```

Calling the `describe` method on the `'species'` column confirms that it contains three unique values. Here, we know in advance of working with this data that there are three classes to which the samples belong, though this is not always the case in unsupervised machine learning.

C

lick here to view code image

```
In [16]: iris_df['species'].describe()
Out[16]:
count         150
unique          3
top        setosa
freq           50
Name: species, dtype: object
```

14.7.3 Visualizing the Dataset with a Seaborn `pairplot`

Let's visualize the features in this dataset. One way to learn more about your data is to see how the features relate to one another. The dataset has four features. We cannot graph one against the other three in a single graph. However, we can plot pairs of features against one another. Snippet `[20]` uses Seaborn function **`pairplot`** to create a grid of graphs plotting each feature against itself and the other specified features:

lick here to view code image

```
In [17]: import seaborn as sns

In [18]: sns.set(font_scale=1.1)

In [19]: sns.set_style('whitegrid')

In [20]: grid = sns.pairplot(data=iris_df,   vars=iris_df.columns[0:4],
    ...:     hue='species')
    ...:
```

The keyword arguments are:

- `data`—The `DataFrame` [3] containing the data to plot.

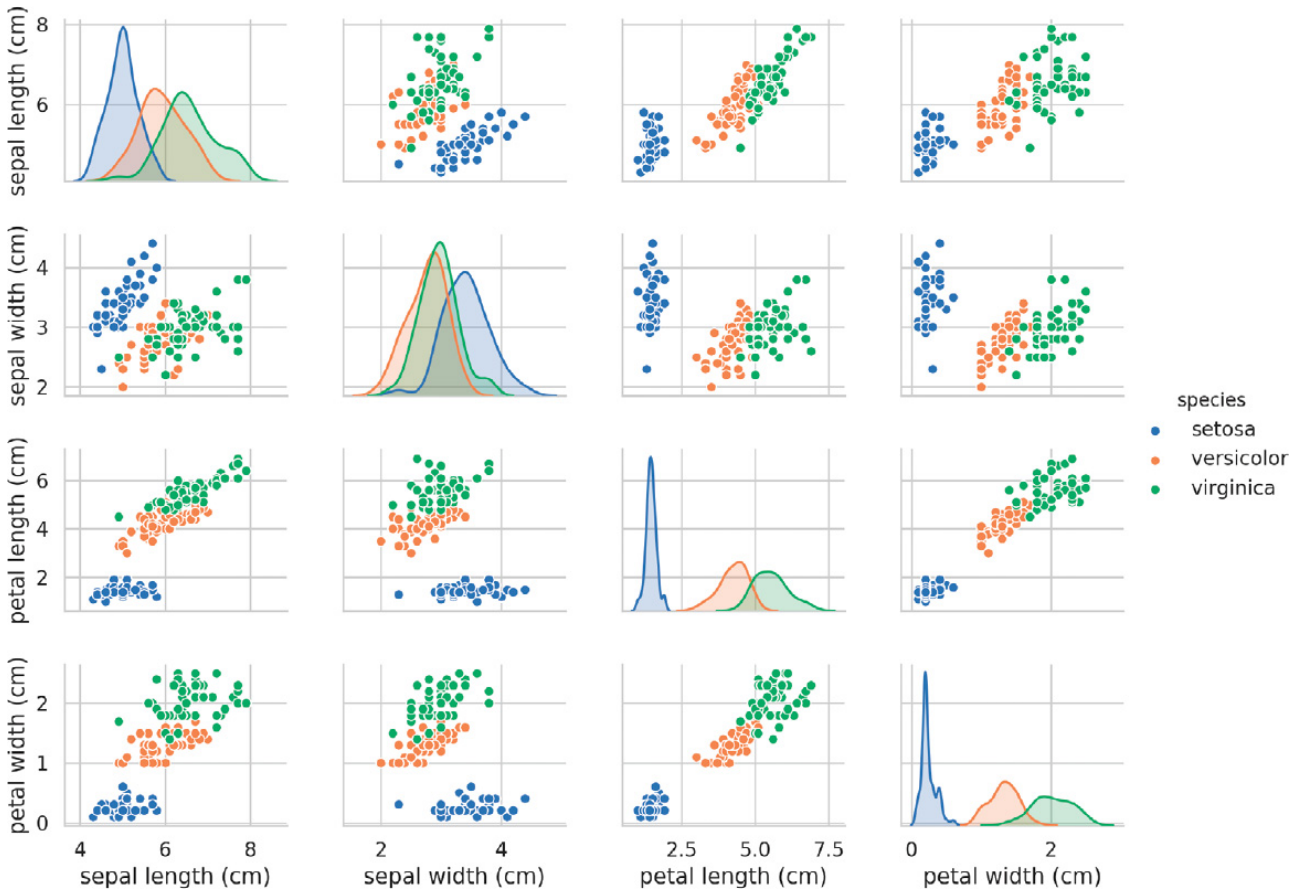
  [3] This also may be a two-dimensional array or list.

- `vars`—A sequence containing the names of the variables to plot. For a `DataFrame`, these are the names of the columns to plot. Here, we use the first four `DataFrame` columns, representing the sepal length, sepal width, petal length and petal width, respectively.

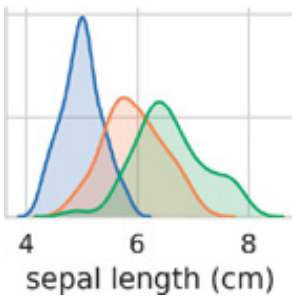
- `hue`—The `DataFrame` column that's used to determine colors of the plotted data. In this case, we'll color the data by *Iris* species.

C

he preceding call to `pairplot` produces the following 4-by-4 grid of graphs:



The graphs along the top-left-to-bottom-right diagonal, show the **distribution** of just the feature plotted in that column, with the range of values (left-to-right) and the number of samples with those values (top-to-bottom). Consider the sepal-length distributions:



The tallest shaded area indicates that the range of sepal length values (shown along the *x*-axis) for *Iris setosa* is approximately 4–6 centimeters and that most *Iris setosa* samples are in the middle of that range (approximately 5 centimeters). Similarly, the rightmost shaded area indicates that the range of sepal length values for *Iris virginica* is approximately 4–8.5 centimeters and that the majority of *Iris virginica* samples have sepal length values between 6 and 7 centimeters.

The other graphs in a column show scatter plots of the other features against the feature on the *x*-axis. In the first column, the other three graphs plot the sepal width, petal length and petal width, respectively, along the *y*-axis and the sepal length along

T

the *x*-axis.

When you run this code, you'll see in the full color output that using separate colors for each *Iris* species shows how the species relate to one another on a feature-by-feature basis. Interestingly, all the scatter plots clearly separate the Iris *setosa* blue dots from the other species' orange and green dots, indicating that *Iris setosa* is indeed in a "class by itself." We also can see that the other two species can sometimes be confused with one another, as indicated by the overlapping orange and green dots. For example, if you look at the scatter plot for sepal width vs. sepal length, you'll see the *Iris versicolor* and *Iris virginica* dots are intermixed. This indicates that it would be difficult to distinguish between these two species if we had only the sepal measurements available to us.

Displaying the `pairplot` in One Color

If you remove the `hue` keyword argument, `pairplot` function uses only one color to plot all the data because it does not know how to distinguish the species:

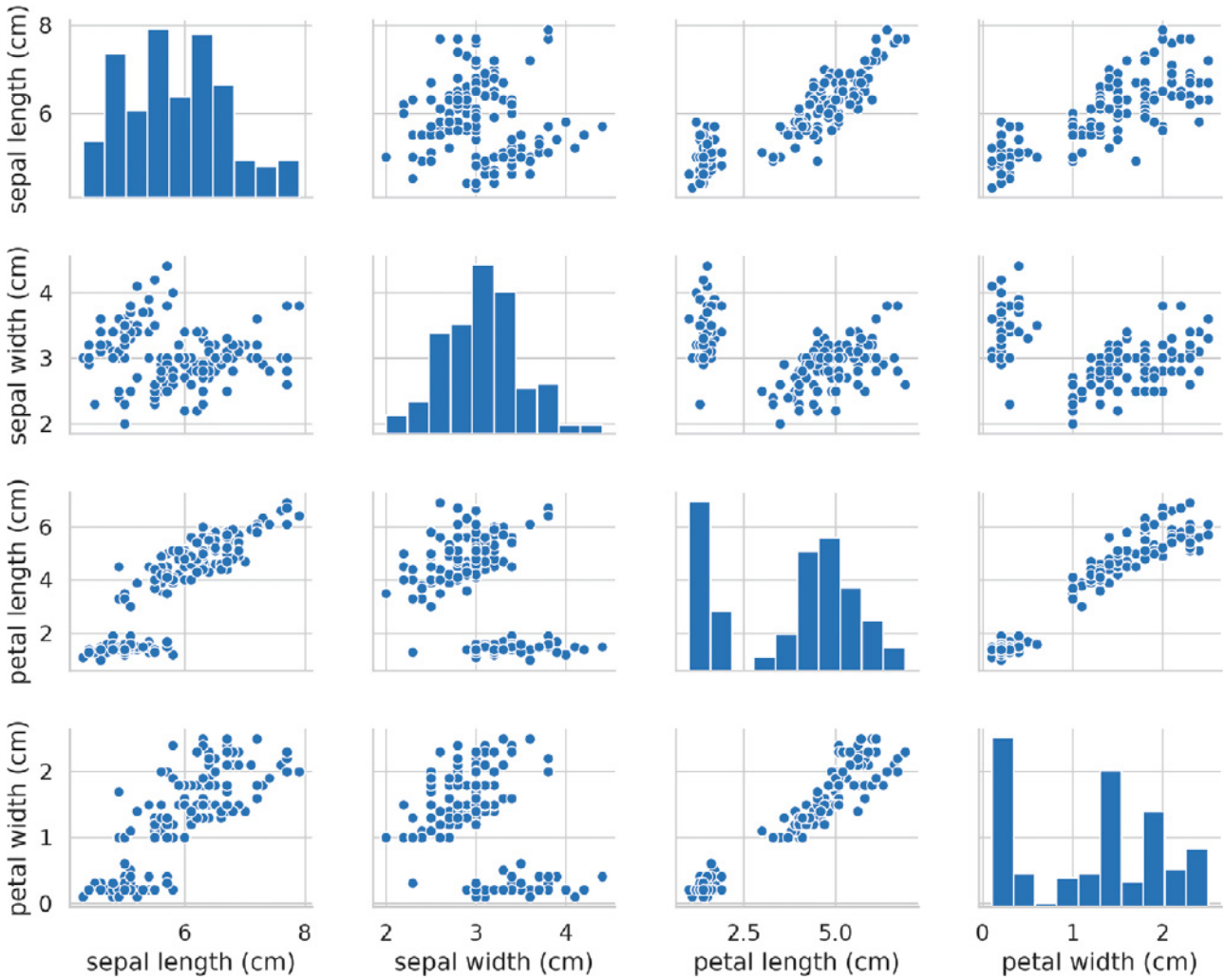
lick here to view code image

```
In [21]: grid = sns.pairplot(data=iris_df,   vars=iris_df.columns[0:4])
```

As you can see in the resulting pair plot on the next page, in this case, the graphs along the diagonal are histograms showing the distributions of all the values for that feature, regardless of the species. As you study each scatter plot, it appears that there may be only *two* distinct clusters, even though for this dataset we know there are *three* species. If you do not know the number of clusters in advance, you might ask a **domain expert** who is thoroughly familiar with the data. Such a person might know that there are three species in the dataset, which would be valuable information as we try to perform machine learning on the data.

The `pairplot` diagrams work well for a *small number of features* or a subset of features so that you have a small number of rows and columns, and for a relatively small number of samples so you can see the data points. As the number of features and samples increases, each scatter plot quickly becomes too small to read. For larger datasets, you may choose to plot a subset of the features and potentially a randomly selected subset of the samples to get a feel for your data.

C



14.7.4 Using a `KMeans` Estimator

In this section, we'll use k-means clustering via scikit-learn's **`KMeans` estimator** (from the **`sklearn.cluster`** module) to place each sample in the Iris dataset into a cluster. As with the other estimators you've used, the `KMeans` estimator hides from you the algorithm's complex mathematical details, making it straightforward to use.

Creating the Estimator

Let's create the `KMeans` object:

lick here to view code image

```
In [22]: from sklearn.cluster import KMeans

In [23]: kmeans = KMeans(n_clusters=3,    random_state=11)
```

The keyword argument `n_clusters` specifies the k-means clustering algorithm's hyperparameter *k*, which `KMeans` requires to calculate the clusters and label each sample. When you train a `KMeans` estimator, the algorithm calculates for each cluster a centroid representing the cluster's center data point.

C

The default value for the `n_clusters` parameter is `8`. Often, you'll rely on domain experts knowledgeable about the data to help choose an appropriate *k* value. However, with hyperparameter tuning, you can estimate the appropriate *k*, as we'll do later. In this case, we know there are three species, so we'll use `n_clusters=3` to see how well `KMeans` does in labeling the Iris samples. Once again, we used the `random_state` keyword argument for reproducibility.

Fitting the Model

Next, we'll train the estimator by calling the `KMeans` object's `fit` method. This step performs the k-means algorithm discussed earlier:

lick here to view code image

```
In [24]: kmeans.fit(iris.data)
Out[24]:
KMeans(algorithm='auto', copy_x=True, init='k-means++',    max_iter=300,
    n_clusters=3, n_init=10, n_jobs=None,    precompute_distances='auto',
    random_state=11, tol=0.0001, verbose=0)
```

As with the other estimator's, the `fit` method returns the estimator object and IPython displays its string representation. You can see the `KMeans` default arguments at:

```
ttps://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.h
```

hen the training completes, the `KMeans` object contains:

- A `labels_` array with values from `0` to `n_clusters - 1` (in this example, 0–2), indicating the clusters to which the samples belong.
- A `cluster_centers_` array in which each row represents a centroid.

Comparing the Computer Cluster Labels to the Iris Dataset's Target Values

Because the Iris dataset is labeled, we can look at its `target` array values to get a sense of how well the k-means algorithm clustered the samples for the three *Iris* species. With unlabeled data, we'd need to depend on a domain expert to help evaluate whether the predicted classes make sense.

W

In this dataset, the first 50 samples are *Iris setosa*, the next 50 are *Iris versicolor*, and the last 50 are *Iris virginica*. The Iris dataset's `target` array represents these with the values 0–2. If the `KMeans` estimator chose the clusters perfectly, then each group of 50 elements in the estimator's `labels_` array should have a distinct label. As you study the results below, note that the `KMeans` estimator uses the values 0 through $k - 1$ to label clusters, but these are not related to the Iris dataset's `target` array.

Let's use slicing to see how each group of 50 Iris samples was clustered. The following snippet shows that the first 50 samples were all placed in cluster `1`:

lick here to view code image

```
In [25]: print(kmeans.labels_[0:50])
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
```

The next 50 samples should be placed into a second cluster. The following snippet shows that most were placed in cluster `0`, but two samples were placed in cluster `2`:

lick here to view code image

```
In [26]: print(kmeans.labels_[50:100])
[0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

Similarly, the last 50 samples should be placed into a third cluster. The following snippet shows that many of these samples were placed in cluster `2`, but 14 of the samples were placed in cluster `0`, indicating that the algorithm thought they belonged to a different cluster:

lick here to view code image

```
In [27]: print(kmeans.labels_[100:150])
[2 0 2 2 2 2 0 2 2 2 2 2 2 0 0 2 2 2 2 0 2 0 2 0 2 2 0 0 2 2 2 2 2 0 2
 2 2 0 2 2 2 0 2 2 2 0 2 2 0]
```

The results of these three snippets confirm what we saw in the `pairplot` diagrams earlier in this section—that *Iris setosa* is "in a class by itself" and that there is some

C

confusion between *Iris versicolor* and *Iris virginica*.

14.7.5 Dimensionality Reduction with Principal Component Analysis

Next, we'll use the **`PCA` estimator** (from the **`sklearn.decomposition`** module) to perform dimensionality reduction. This estimator uses an algorithm called principal component analysis [4] to analyze a dataset's features and reduce them to the specified number of dimensions. For the Iris dataset, we first tried the `TSNE` estimator shown earlier but did not like the results we were getting. So we switched to `PCA` for the following demonstration.

[4] The algorithms details are beyond this books scope. For more information, see `ttps://scikit-learn.org/stable/modules/decomposition.html#pca`.

Creating the PCA Object

Like the `TSNE` estimator, a `PCA` estimator uses the keyword argument `n_components` to specify the number of dimensions:

lick here to view code image

```
In [28]: from   sklearn.decomposition import PCA

In [29]: pca = PCA(n_components=2,   random_state=11)
```

Transforming the Iris Dataset's Features into Two Dimensions

Let's train the estimator and produce the reduced data by calling the PCA estimator's methods **`fit`** and **`transform`** methods:

lick here to view code image

```
In [30]: pca.fit(iris.data)
Out[30]:
PCA(copy=True, iterated_power='auto', n_components=2,   random_state=11,
  svd_solver='auto', tol=0.0, whiten=False)

In [31]: iris_pca = pca.transform(iris.data)
```

When the method completes its task, it returns an array with the same number of rows as `iris.data`, but only two columns. Let's confirm this by checking `iris_pca`'s

shape:

lick here to view code image

```
In [32]: iris_pca.shape
Out[32]: (150, 2)
```

Note that we *separately* called the `PCA` estimator's `fit` and `transform` methods, rather than `fit_transform`, which we used with the `TSNE` estimator. In this example, we're going to *reuse* the trained estimator (produced with `fit`) to perform a second `transform` to reduce the cluster centroids from four dimensions to two. This will enable us to plot the centroid locations on each cluster.

Visualizing the Reduced Data

Now that we've reduced the original dataset to only two dimensions, let's use a scatter plot to display the data. In this case, we'll use Seaborn's `scatterplot` function. First, let's transform the reduced data into a `DataFrame` and add a species column that we'll use to determine the dot colors:

lick here to view code image

```
In [33]: iris_pca_df = pd.DataFrame(iris_pca,
    ...:                            columns=['Component1', 'Component2'])
    ...:

In [34]: iris_pca_df['species']   = iris_df.species
```

Next, let's scatterplot the data in Seaborn:

lick here to view code image

```
In [35]: axes = sns.scatterplot(data=iris_pca_df, x='Component1',
    ...:     y='Component2',   hue='species', legend='brief',
    ...:     palette='cool')
    ...:
```

Each centroid in the `KMeans` object's `cluster_centers_` array has the *same* number of features as the original dataset (four in this case). To plot the centroids, we must reduce their dimensions. You can think of a centroid as the "average" sample in its cluster. So each centroid should be transformed using the same `PCA` estimator we used

C

to reduce the other samples in that cluster:

lick here to view code image

```
In [36]: iris_centers =   pca.transform(kmeans.cluster_centers_)
```

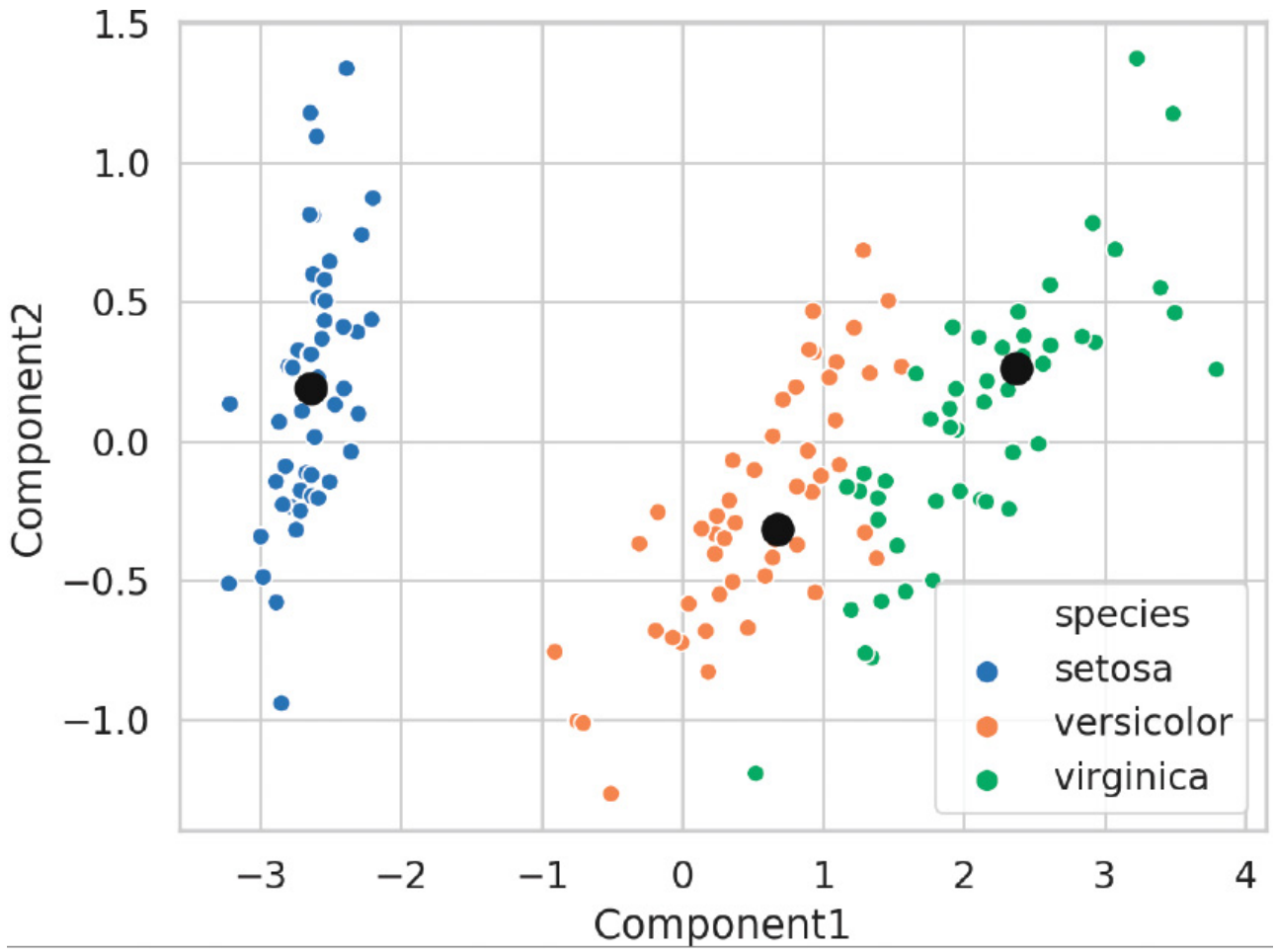
Now, we'll plot the centroids of the three clusters as larger black dots. Rather than transform the `iris_centers` array into a `DataFrame` first, let's use Matplotlib's `scatter` function to plot the three centroids:

lick here to view code image

```
In [37]: import   matplotlib.pyplot as plt

In [38]: dots = plt.scatter(iris_centers[:,0], iris_centers[:,1],
    ...:                    s=100,   c='k')
    ...:
```

The keyword argument `s=100` specifies the size of the plotted points, and the keyword argument `c='k'` specifies that the points should be displayed in black.



C

14.7.6 Choosing the Best Clustering Estimator

As we did in the classification and regression case studies, let's run multiple clustering algorithms and see how well they cluster the three species of Iris flowers. Here we'll attempt to cluster the Iris dataset's samples using the `kmeans` object we created earlier [5] and objects of scikit-learn's `DBSCAN`, `MeanShift`, `SpectralClustering` and `AgglomerativeClustering` estimators. Like `KMeans`, you specify the number of clusters in advance for the `SpectralClustering` and `AgglomerativeClustering` estimators:

[5]Were running `KMeans` here on the *small* Iris dataset. If you experience performance problems with `KMeans` on larger datasets, consider using the `MiniBatchKMeans` estimator. The scikit-learn documentation indicates that `MiniBatchKMeans` is faster on large datasets and the results are almost as good.

lick here to view code image

```
In [39]: from sklearn.cluster import DBSCAN, MeanShift,\
    ...:     SpectralClustering, AgglomerativeClustering

In [40]: estimators = {
    ...:     'KMeans': kmeans,
    ...:     'DBSCAN':   DBSCAN(),
    ...:     'MeanShift':   MeanShift(),
    ...:     'SpectralClustering':   SpectralClustering(n_clusters=3),
    ...:     'AgglomerativeClustering':
    ...:         AgglomerativeClustering(n_clusters=3)
    ...: }
```

Each iteration of the following loop calls one estimator's `fit` method with `iris.data` as an argument, then uses NumPy's `unique` function to get the cluster labels and counts for the three groups of 50 samples and displays the results. Recall that for the `DBSCAN` and `MeanShift` estimators, we did *not* specify the number of clusters in advance. Interestingly, `DBSCAN` correctly predicted three clusters (labeled `-1`, `0` and `1`), though it placed 84 of the 100 *Iris virginica* and *Iris versicolor* samples in the same cluster. The `MeanShift` estimator, on the other hand, predicted only two clusters (labeled as `0` and `1`), and placed 99 of the 100 *Iris virginica* and *Iris versicolor* samples in the same cluster:

lick here to view code image

```
In [41]: import numpy as np
```

C

```
In [42]: for name, estimator in estimators.items():
    ...:     estimator.fit(iris.data)
    ...:     print(f'\n{name}:')
    ...:     for i in range(0, 101, 50):
    ...:         labels, counts = np.unique(
    ...:             estimator.labels_[i:i+50], return_counts=True)
    ...:         print(f'{i}-{i+50}:')
    ...:         for label,   count in zip(labels, counts):
    ...:             print(f'     label={label}, count={count}')
    ...:

KMeans:
0-50:
   label=1, count=50
50-100:
   label=0, count=48
   label=2, count=2
100-150:
   label=0, count=14
   label=2, count=36

DBSCAN:
0-50:
   label=-1, count=1
   label=0, count=49
50-100:
   label=-1, count=6
   label=1, count=44
100-150:
   label=-1, count=10
   label=1, count=40

MeanShift:
0-50:
   label=1, count=50
50-100:
   label=0, count=49
   label=1, count=1
100-150:
   label=0, count=50

SpectralClustering:
0-50:
   label=2, count=50
50-100:
   label=1, count=50
100-150:
   label=0, count=35
   label=1, count=15

AgglomerativeClustering:
0-50:
   label=1, count=50
```

I

```
   50-100:
      label=0, count=49
      label=2, count=1
   100-150:
      label=0, count=15
      label=2, count=35
```

Though these algorithms label every sample, the labels simply indicate the clusters. What do you do with the cluster information once you have it? If your goal is to use the data in supervised machine learning, typically you'd study the samples in each cluster to try to determine how they're related and label them accordingly. As we'll see in the next chapter, unsupervised learning is commonly used in deep-learning applications. Some examples of unlabeled data processed with unsupervised learning include tweets from Twitter, Facebook posts, videos, photos, news articles, customers' product reviews, viewers' movie reviews and more.

14.8 WRAP-UP

In this chapter we began our study of machine learning, using the popular scikit-learn library. We saw that machine learning is divided into two types. Supervised machine learning, which works with labeled data and unsupervised machine learning which works with unlabeled data. Throughout this chapter, we continued emphasizing visualizations using Matplotlib and Seaborn, particularly for getting to know your data.

We discussed how scikit-learn conveniently packages machine-learning algorithms as estimators. Each is encapsulated so you can create your models quickly with a small amount of code, even if you don't know the intricate details of how these algorithms work.

We looked at supervised machine learning with classification, then regression. We used one of the simplest classification algorithms, k-nearest neighbors, to analyze the Digits dataset bundled with scikit-learn. You saw that classification algorithms predicts the classes to which samples belong. Binary classification uses two classes (such as "spam" or "not spam") and multi-classification uses more than two classes (such as the 10 classes in the Digits dataset).

We performed the steps of a typical machine-learning case study, including loading the dataset, exploring the data with pandas and visualizations, splitting the data for training and testing, creating the model, training the model and making predictions. We discussed why you should partition your data into a training set and a testing set. You saw ways to evaluate a classification estimator's accuracy via a confusion matrix

nd a classification report.

We mentioned that it's difficult to know in advance which model(s) will perform best on your data, so you typically try many models and pick the one that performs best. We showed that it's easy to run multiple estimators. We also used hyperparameter tuning with k-fold cross-validation to choose the best value of k for the k-NN algorithm.

We revisited the time series and simple linear regression example from hapter 10's Intro to Data Science section, this time implementing it using a scikit-learn `LinearRegression` estimator. Next, we used a `LinearRegression` estimator to perform multiple linear regression with the California Housing dataset that's bundled with scikit-learn. You saw that the `LinearRegression` estimator, by default, uses all the numerical features in a dataset to make more sophisticated predictions than you can with simple linear regression. Again, we ran multiple scikit-learn estimators to compare how they performed and choose the best one.

Next, we introduced an unsupervised machine learning and mentioned that it's typically accomplished with clustering algorithms. We used introduced dimensionality reduction (with scikit-learn's `TSNE` estimator) and used it to compress the Digits dataset's 64 features down to two for visualization purposes. This enabled us to see the clustering of the digits data.

We presented one of the simplest unsupervised machine learning algorithms, k-means clustering, and demonstrated clustering on the Iris dataset that's also bundled with scikit-learn. We used dimensionality reduction (with scikit-learn's `PCA` estimator) to compress the Iris dataset's four features to two for visualization purposes to show the clustering of the three *Iris* species in the dataset and their centroids. Finally, we ran multiple clustering estimators to compare their ability to label the Iris dataset's samples into three clusters.

In the next chapter, we'll continue our study of machine learning technologies with deep learning. We'll tackle some fascinating and challenging problems.

15. Deep Learning

Objectives

In this chapter you'll:

- Understand what a neural network is and how it enables deep learning.
- Create Keras neural networks.
- Understand Keras layers, activation functions, loss functions and optimizers.
- Use a Keras convolutional neural network (CNN) trained on the MNIST dataset to recognize handwritten digits.
- Use a Keras recurrent neural network (RNN) trained on the IMDb dataset to perform binary classification of positive and negative movie reviews.
- Use TensorBoard to visualize the progress of training deep-learning networks.
- Learn which pretrained neural networks come with Keras.
- Understand the value of using models pretrained on the massive ImageNet dataset for computer vision apps.

Outline

15.1 INTRODUCTION

One of AI's most exciting areas is **deep learning**, a powerful subset of machine learning that has produced impressive results in computer vision and many other areas over the last few years. The availability of big data, significant processor power, faster Internet speeds and advancements in parallel computing hardware and software are making it possible for more organizations and individuals to pursue resource-intensive deep-learning solutions.

Keras and TensorFlow

In the previous chapter, Scikit-learn enabled you to define machine-learning models conveniently with one statement. Deep learning models require more sophisticated setups, typically connecting multiple objects, called **layers**. We'll build our deep learning models with **Keras**, which offers a friendly interface to Google's **TensorFlow**—the most widely used deep-learning library. [1] François Chollet of the Google Mind team developed Keras to make deep-learning capabilities more accessible. His book *Deep Learning with Python* is a must read. [2] Google has thousands of TensorFlow and Keras projects underway internally

nd that number is growing quickly. [3] , [4]

[1] Keras also serves as a friendlier interface to Microsofts *CNTK* and the Université de Montréals *Theano*- (which ceased development in 2017). Other popular deep learning frameworks include Caffe (`ttp://caffe.berkeleyvision.org/`), Apache MXNet (`ttps://mxnet.apache.org/`) and PyTorch (`ttps://pytorch.org/`).

[2] Chollet, François. *Deep Learning with Python*. Shelter Island, NY: Manning Publications, 2018.

[3] `ttp://theweek.com/speedreads/654463/google-more-than-1000-rtificial-intelligence-projects-works`.

[4] `ttps://www.zdnet.com/article/google-says-exponential-growth-of-i-is-changing-nature-of-compute/`.

Models

Deep learning models are complex and require an extensive mathematical background to understand their inner workings. As we've done throughout the book, we'll avoid heavy mathematics here, preferring English explanations.

Keras is to deep learning as Scikit-learn is to machine learning. Each encapsulates the sophisticated mathematics, so developers need only define, parameterize and manipulate objects. With Keras, you build your models from *pre-existing* components and quickly parameterize those components to your unique requirements. This is what we've been referring to as *object-based programming* throughout the book.

Experiment with Your Models

Machine learning and deep learning are empirical rather than theoretical fields. You'll experiment with many models, tweaking them in various ways until you find the models that perform best for your applications. Keras facilitates such experimentation.

Dataset Sizes

Deep learning works well when you have lots of data, but it also can be effective for smaller datasets when combined with techniques like transfer learning [5] , [6] and data augmentation [7] , [8] . Transfer learning uses existing knowledge from a previously trained model as the foundation for a new model. Data augmentation adds data to a dataset by deriving new data from existing data. For example, in an image dataset, you might rotate the images left and right so the model can learn about objects in different orientations. In general, though, the more data you have, the better you'll be able to train a deep learning model.

[5] `ttps://towardsdatascience.com/transfer-learning-from-pre-trained-odels-f2393f124751`.

[6] `ttps://medium.com/nanonets/nanonets-how-to-use-deep-learning-hen-you-have-limited-data-f68c0b512cab`.

[7] `ttps://towardsdatascience.com/data-augmentation-and-images-`

aca9bd0dbe8.

[8] ttps://medium.com/nanonets/how-to-use-deep-learning-when-you-ave-limited-data-part-2-data-augmentation-c26971dc8ced.

rocessing Power

Deep learning can require significant processing power. Complex models trained on big-data datasets can take hours, days or even more to train. The models we present in this chapter can be trained in minutes to just less than an hour on computers with conventional CPUs. You'll need only a reasonably current personal computer. We'll discuss the special high-performance hardware called GPUs (Graphics Processing Units) and TPUs (Tensor Processing Units) developed by NVIDIA and Google to meet the extraordinary processing demands of edge-of-the-practice deep-learning applications.

Bundled Datasets

Keras comes packaged with some popular datasets. You'll work with two of these datasets in the chapter's examples. You can find many Keras studies online for each of these datasets, including ones that take different approaches.

In the "Machine Learning" chapter, you worked with Scikit-learn's Digits dataset, which contained 1797 handwritten-digit images that were selected from the much larger MNIST dataset (60,000 training images and 10,000 test images). [9] In this chapter you'll work with the full MNIST dataset. You'll build a Keras *convolutional neural network* (CNN or convnet) model that will achieve high performance recognizing digit images in the test set. Convnets are especially appropriate for computer vision tasks, such as recognizing handwritten digits and characters or recognizing objects (including faces) in images and videos. You'll also work with a Keras *recurrent neural network*. In that example, you'll perform sentiment analysis using the IMDb Movie reviews dataset, in which the reviews in the training and testing sets are labeled as positive or negative.

[9] The MNIST Database. MNIST Handwritten Digit Database, Yann LeCun, Corinna Cortes and Chris Burges. ttp://yann.lecun.com/exdb/mnist/.

Future of Deep Learning

Newer automated deep learning capabilities are making it even easier to build deep-learning solutions. These include Auto-Keras [10] from Texas A&M University's DATA Lab, Baidu's EZDL [11] and Google's AutoML [12].

[10] ttps://autokeras.com/.

[11] ttps://ai.baidu.com/ezdl/.

[12] ttps://cloud.google.com/automl/.

15.1.1 Deep Learning Applications

Deep learning is being used in a wide range of applications, such as:

- Game playing
- Computer vision: Object recognition, pattern recognition, facial recognition
- Self-driving cars
- Robotics
- Improving customer experiences
- Chatbots
- Diagnosing medical conditions
- Google Search
- Facial recognition
- Automated image captioning and video closed captioning
- Enhancing image resolution
- Speech recognition
- Language translation
- Predicting election results
- Predicting earthquakes and weather
- Google Sunroof to determine whether you can put solar panels on your roof
- Generative applications—Generating original images, processing existing images to look like a specified artist's style, adding color to black-and-white images and video, creating music, creating text (books, poetry) and much more.

15.1.2 Deep Learning Demos

Check out these four deep-learning demos and search online for lots more, including practical applications like we mentioned in the preceding section:

- DeepArt.io—Turn a photo into artwork by applying an art style to the photo. `ttps://deepart.io/`.
- DeepWarp Demo—Analyzes a person's photo and makes the person's eyes move in different directions. `ttps://sites.skoltech.ru/sites/compvision_wiki/static_pages/projects/dee`
- Image-to-Image Demo—Translates a line drawing into a picture. `ttps://affinelayer.com/pixsrv/`.
- Google Translate Mobile App (download from an app store to your smartphone)—

h

ranslate text in a photo to another language (e.g., take a photo of a sign or a restaurant menu in Spanish and translate the text to English).

15.1.3 Keras Resources

Here are some resources you might find valuable as you study deep learning:

- To get your questions answered, go to the Keras team's slack channel at `ttps://kerasteam.slack.com`.
- For articles and tutorials, visit `ttps://blog.keras.io`.
- The Keras documentation is at `ttp://keras.io`.
- If you're looking for term projects, directed study projects, capstone course projects or thesis topics, visit arXiv (pronounced "archive," where the X represents the Greek letter "chi") at `ttps://arXiv.org`. People post their research papers here in parallel with going through peer review for formal publication, hoping for fast feedback. So, this site gives you access to extremely current research.

15.2 KERAS BUILT-IN DATASETS

Here are some of Keras's datasets (from the module **`tensorflow.keras.datasets`**[3]) for practicing deep learning. We'll use a couple of these in the chapter's examples:

[3]In the standalone Keras library, the module names begin with `keras` rather than `tensorflow.keras`.

- **MNIST database of handwritten digits**[4]—Used for classifying handwritten digit images, this dataset contains 28-by-28 grayscale digit images labeled as 0 through 9 with 60,000 images for training and 10,000 for testing. We use this dataset in ection 15.6, where we study convolutional neural networks.

  [4]The MNIST Database. MNIST Handwritten Digit Database, Yann LeCun, Corinna Cortes and Chris Burges. `ttp://yann.lecun.com/exdb/mnist/`.

- **Fashion-MNIST database of fashion articles**[5]—Used for classifying clothing images, this dataset contains 28-by-28 grayscale images of clothing labeled in 10 categories [6] with 60,000 for training and 10,000 for testing. Once you build a model for use with MNIST, you can reuse that model with Fashion-MNIST by changing a few statements.

- **IMDb Movie reviews**[7]—Used for sentiment analysis, this dataset contains reviews labeled as positive (1) or negative (0) sentiment with 25,000 reviews for training and 25,000 for testing. We use this dataset in ection 15.9, where we study recurrent neural networks.

  [5]Han Xiao and Kashif Rasul and Roland Vollgraf, Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms, arXiv, cs.LG/1708.07747.

[6] ttps://keras.io/datasets/#fashion-mnist-database-of-fashion-rticles.

[7]Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. (2011). Learning Word Vectors for Sentiment Analysis. The 49th Annual Meeting of the Association for Computational Linguistics (ACL 2011).

- **CIFAR10**[8] **small image classification**—Used for small-image classification, this dataset contains 32-by-32 color images labeled in 10 categories with 50,000 images for training and 10,000 for testing.

[8] ttps://www.cs.toronto.edu/~kriz/cifar.html.

- **CIFAR100**[9] **small image classification**—Also, used for small-image classification, this dataset contains 32-by-32 color images labeled in 100 categories with 50,000 images for training and 10,000 for testing.

[9] ttps://www.cs.toronto.edu/~kriz/cifar.html.

15.3 CUSTOM ANACONDA ENVIRONMENTS

Before running this chapter's examples, you'll need to install the libraries we use. In this chapter's examples, we'll use the TensorFlow deep-learning library's version of Keras.[0] At the time of this writing, TensorFlow does not yet support Python 3.7. So, you'll need Python 3.6.x to execute this chapter's examples. We'll show you how to set up a *custom environment* for working with Keras and TensorFlow.

[0]Theres also a standalone version that enables you to choose between TensorFlow, Microsofts *CNTK* or the Université de Montréals *Theano* (which ceased development in 2017).

Environments in Anaconda

The Anaconda Python distribution makes it easy to create custom **environments**. These are separate configurations in which you can install different libraries and different library versions. This can help with *reproducibility* if your code depends on specific Python or library versions.[1]

[1]In the next chapter, well introduce Docker as another reproducibility mechanism and as a convenient way to install complex environments for use on your local computer.

The default environment in Anaconda is called the *base environment*. This is created for you when you install Anaconda. All the Python libraries that come with Anaconda are installed into the base environment and, unless you specify otherwise, any additional libraries you install also are placed there. Custom environments give you control over the specific libraries you wish to install for your specific tasks.

Creating an Anaconda Environment

The `conda create` **command** creates an environment. Let's create a TensorFlow

environment and name it `tf_env` (you can name it whatever you like). Run the following command in your Terminal, shell or Anaconda Command Prompt: [2, 3]

[2]Windows users should run the Anaconda Command Prompt as Administrator,

[3]If you have a computer with an NVIDIA GPU thats compatible with TensorFlow, you can replace the `tensorflow` library with `tensorflow-gpu` to get better performance. For more information, see `ttps://www.tensorflow.org/install/gpu`. Some AMD GPUs also can be used with TensorFlow: `ttp://timdettmers.com/2018/11/05/which-gpu-or-deep-learning/`.

lick here to view code image

```
conda create -n tf_env tensorflow anaconda ipython jupyterlab scikit-learn matp
```

his will determine the listed libraries' dependencies, then display all the libraries that will be installed in the new environment. There are many dependencies, so this may take a few minutes. When you see the prompt:

```
Proceed ([y]/n)?
```

press *Enter* to create the environment and install the libraries. [4]

[4]When we created our custom environment, conda installed Python 3.6.7, which was the most recent Python version compatible with the `tensorflow` library.

Activating an Alternate Anaconda Environment

To use a custom environment, execute the **`conda activate` command**:

```
conda activate tf_env
```

This affects only the current Terminal, shell or Anaconda Command Prompt. When a custom environment is activated and you install more libraries, they become part of the activated environment, not the base environment. If you open separate Terminals, shells or Anaconda Command Prompts, they'll use Anaconda's base environment by default.

Deactivating an Alternate Anaconda Environment

When you're done with a custom environment, you can return to the base environment in the current Terminal, shell or Anaconda Command Prompt by executing:

```
conda deactivate
```

Jupyter Notebooks and JupyterLab

This chapter's examples are provided only as Jupyter Notebooks, which will make it easier for you to experiment with the examples. You can tweak the options we present and reexecute

he notebooks. For this chapter, you should launch JupyterLab from the `ch15` examples folder (as discussed in ection 1.5.3).

15.4 NEURAL NETWORKS

Deep learning is a form of machine learning that uses artificial neural networks to learn. An **artificial neural network** (or just neural network) is a software construct that operates similarly to how scientists believe our brains work. Our biological nervous systems are controlled via *neurons* [5] that communicate with one another along pathways called *synapses* [6]. As we learn, the specific neurons that enable us to perform a given task, like walking, communicate with one another more efficiently. These neurons *activate* anytime we need to walk. [7]

[5] `ttps://en.wikipedia.org/wiki/Neuron`.

[6] `ttps://en.wikipedia.org/wiki/Synapse`.

[7] `ttps://www.sciencenewsforstudents.org/article/learning-rewires-rain`.

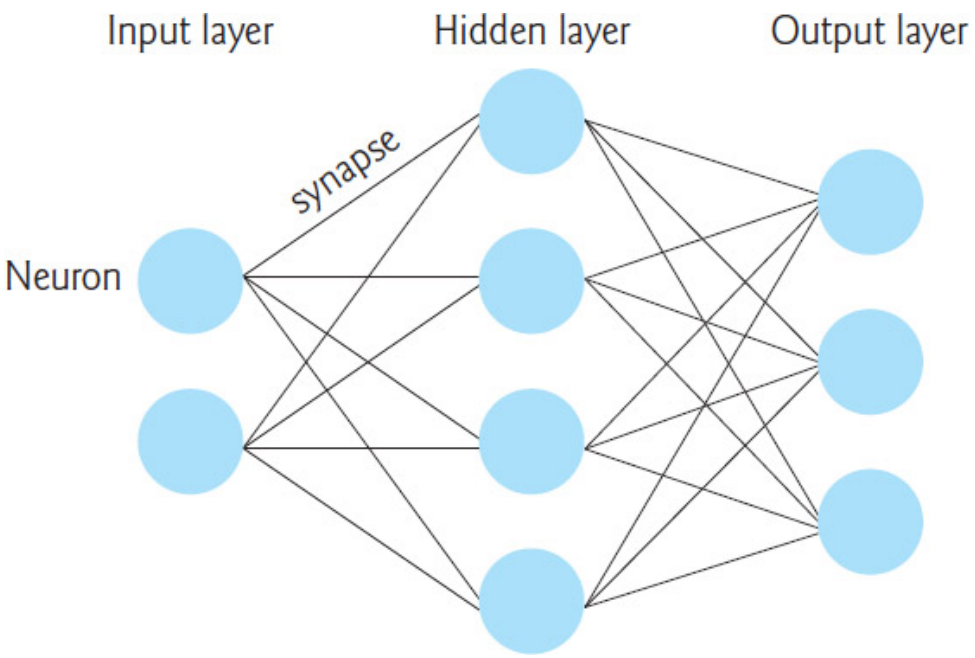
Artificial Neurons

In a neural network, interconnected **artificial neurons** simulate the human brain's neurons to help the network learn. The connections between specific neurons are reinforced during the learning process with the goal of achieving a specific result. In **supervised deep learning**—which we'll use in this chapter—we aim to predict the target labels supplied with data samples. To do this, we'll train a general neural network model that we can then use to make predictions on unseen data. [8]

[8] As in machine learning, you can create *unsupervised* deep learning networksthese are beyond this chapters scope.

Artificial Neural Network Diagram

The following diagram shows a three-*layer* neural network. Each circle represents a neuron, and the lines between them simulate the synapses. The output of a neuron becomes the input of another neuron, hence the term neural network. This particular diagram shows a **fully connected network**—every neuron in a given layer is connected to *all* the neurons in the next layer:



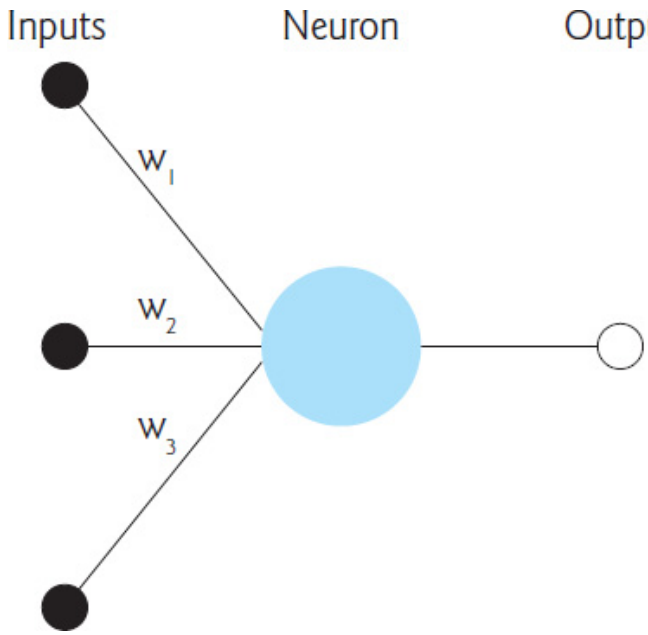
Learning Is an Iterative Process

When you were a baby, you did not learn to walk instantaneously. You learned that process *over time* with repetition. You built up the smaller components of the movements that enabled you to walk—learning to stand, learning to balance to remain standing, learning to lift your foot and move it forward, etc. And you got feedback from your environment. When you walked successfully your parents smiled and clapped. When you fell, you might have bumped your head and felt pain.

Similarly, we train neural networks iteratively over time. Each iteration is known as an **epoch** and processes every sample in the training dataset once. There's no "correct" number of epochs. This is a hyperparameter that may need tuning, based on your training data and your model. The inputs to the network are the features in the training samples. Some layers learn new features from previous layers' outputs and others interpret those features to make predictions.

How Artificial Neurons Decide Whether to Activate Synapses

During the training phase, the network calculates values called **weights** for every connection between the neurons in one layer and those in the next. On a neuron-by-neuron basis, each of its inputs is multiplied by that connection's weight, then the sum of those weighted inputs is passed to the neuron's **activation function**. This function's output determines which neurons to activate based on the inputs—just like the neurons in your brain passing information around in response to inputs coming from your eyes, nose, ears and more. The following diagram shows a neuron receiving three inputs (the black dots) and producing an output (the hollow circle) that would be passed to all or some of neurons in the next layer, depending on the types of the neural network's layers:



The values w_1, w_2 and w_3 are weights. In a new model that you train from scratch, these values are initialized randomly by the model. As the network trains, it tries to minimize the error rate between the network's predicted labels and the samples' actual labels. The error rate is known as the **loss**, and the calculation that determines the loss is called the **loss function**. Throughout training, the network determines the amount that each neuron contributes to the overall loss, then goes back through the layers and adjusts the weights in an effort to minimize that loss. This technique is called **backpropagation**. Optimizing these weights occurs gradually—typically via a process called **gradient descent**.

15.5 TENSORS

Deep learning frameworks generally manipulate data in the form of **tensors**. A "tensor" is basically a multidimensional array. Frameworks like TensorFlow pack all your data into one or more tensors, which they use to perform the mathematical calculations that enable neural networks to learn. These tensors can become quite large as the number of dimensions increases and as the richness of the data increases (for example, images, audios and videos are richer than text). Chollet discusses the types of tensors typically encountered in deep learning: [9]

[9]Chollet, François. *Deep Learning with Python*. ection 2.2. Shelter Island, NY: Manning Publications, 2018.

- **0D (0-dimensional) tensor**—This is one value and is known as a *scalar*.
- **1D tensor**—This is similar to a one-dimensional array and is known as a *vector*. A 1D tensor might represent a sequence, such as hourly temperature readings from a sensor or the words of one movie review.
- **2D tensor**—This is similar to a two-dimensional array and is known as a *matrix*. A 2D tensor could represent a grayscale image in which the tensor's two dimensions are the image's width and height in pixels, and the value in each element is the intensity of that pixel.
- **3D tensor**—This is similar to a three-dimensional array and could be used to represent a

olor image. The first two dimensions would represent the width and height of the image in pixels and the *depth* at each location might represent the red, green and blue (RGB) components of a given pixel's color. A 3D tensor also could represent a *collection* of 2D tensors containing grayscale images.

- **4D tensor**—A 4D tensor could be used to represent a *collection* of color images in 3D tensors. It also could be used to represent one video. Each frame in a video is essentially a color image.

- **5D tensor**—This could be used to represent a collection of 4D tensors containing videos.

A tensor's *shape* typically is represented as a tuple of values in which the number of elements specifies the tensor's number of dimensions and each value in the tuple specifies the size of the tensor's corresponding dimension.

Let's assume we're creating a deep-learning network to identify and track objects in 4K (high-resolution) videos that have 30 frames-per-second. Each frame in a 4K video is 3840-by-2160 pixels. Let's also assume the pixels are presented as red, green and blue components of a color. So *each frame* would be a 3D tensor containing a total of 24,883,200 elements (3840 * 2160 * 3) and each video would be a 4D tensor containing the sequence of frames. If the videos are one minute long, you'd have 44,789,760,000 elements *per tensor*!

Over 600 hours of video are uploaded to YouTube every minute [0] so, in just one minute of uploads, Google could have a tensor containing 1,612,431,360,000,000 elements to use in training deep-learning models—that's *big data*. As you can see, tensors can quickly become *enormous*, so manipulating them efficiently is crucial. This is one of the key reasons that most deep learning is performed on GPUs. More recently Google created TPUs (Tensor Processing Units) that are specifically designed to perform tensor manipulations, executing faster than GPUs.

[0] ttps://www.inc.com/tom-popomaronis/youtube-analyzed-trillions-of-data-points-in-2018-revealing-5-eye-opening-behavioral-tatistics.html.

High-Performance Processors

Powerful processors are needed for real-world deep learning because the size of tensors can be enormous and large-tensor operations can place crushing demands on processors. The processors most commonly used for deep learning are:

- NVIDIA GPUs (Graphics Processing Units)—Originally developed by companies like NVIDIA for computer gaming, GPUs are much faster than conventional CPUs for processing large amounts of data, thus enabling developers to train, validate and test deep-learning models more efficiently—and thus experiment with more of them. GPUs are optimized for the mathematical matrix operations typically performed on tensors, an essential aspect of how deep learning works "under the hood." NVIDIA's Volta Tensor Cores are specifically designed for deep learning. [1, 2] Many NVIDIA GPUs are compatible with TensorFlow, and hence Keras, and can enhance the performance of your deep-learning models. [3]

[1] ttps://www.nvidia.com/en-us/data-center/tensorcore/.

[2] ttps://devblogs.nvidia.com/tensor-core-ai-performance-ilestones/.

[3] ttps://www.tensorflow.org/install/gpu.

- Google TPUs (Tensor Processing Units)—Recognizing that deep learning is crucial to its future, Google developed TPUs (Tensor Processing Units), which they now use in their Cloud TPU service, which "can provide up to 11.5 petaflops of performance in a single pod" [4] (that's 11.5 *quadrillion* floating-point operations per second). Also, TPUs are designed to be especially energy efficient. This is a key concern for companies like Google with already massive computing clusters that are growing exponentially and consuming vast amounts of energy.

[4] ttps://cloud.google.com/tpu/.

15.6 CONVOLUTIONAL NEURAL NETWORKS FOR VISION; MULTI-CLASSIFICATION WITH THE MNIST DATASET

In the "Machine Learning" chapter, we classified handwritten digits using the 8-by-8-pixel, low-resolution images from the Digits dataset bundled with Scikit-learn. That dataset is based on a subset of the higher-resolution MNIST handwritten digits dataset. Here, we'll use MNIST to explore deep learning with a **convolutional neural network** [5] (also called a **convnet** or **CNN**). Convnets are common in computer-vision applications, such as recognizing handwritten digits and characters, and recognizing objects in images and video. They're also used in non-vision applications, such as natural-language processing and recommender systems.

[5] ttps://en.wikipedia.org/wiki/Convolutional_neural_network.

The Digits dataset has only 1797 samples, whereas MNIST has 70,000 labeled digit image samples—60,000 for training and 10,000 for testing. Each sample is a grayscale 28-by-28 pixel image (784 total features) represented as a NumPy array. Each pixel is a value from 0 to 255 representing the intensity (or shade) of that pixel—the Digits dataset uses less granular shading with values from 0 to 16. MNIST's labels are integer values in the range 0 through 9, indicating the digit each image represents.

The machine-learning model you used in the previous chapter produced as its output a digit image's predicted class—an integer in the range 0–9. The convnet model we'll build will perform **probabilistic classification**. [6] For each digit image, the model will output an *array* of 10 probabilities, each indicating the likelihood that the digit belongs to a particular one of the classes 0 through 9. The class with the *highest* probability is the predicted value.

[6] ttps://en.wikipedia.org/wiki/Probabilistic_classification.

Reproducibility in Keras and Deep Learning

We've discussed the importance of *reproducibility* throughout the book. In deep learning, reproducibility is more difficult because the libraries heavily parallelize operations that

perform floating-point calculations. Each time operations execute, they may execute in a different order. This can produce differences in your results. Getting reproducible results in Keras requires a combination of environment settings and code settings that are described in the Keras FAQ:

lick here to view code image

```
ttps://keras.io/getting-started/faq/#how-can-i-obtain-reproducible-results-usi
```

asic Keras Neural Network

A Keras neural network consists of the following components:

- A **network** (also called a **model**)—A sequence of **layers** containing the neurons used to learn from the samples. Each layer's neurons receive inputs, process them (via an *activation function*) and produce outputs. The data is fed into the network via an **input layer** that specifies the dimensions of the sample data. This is followed by **hidden layers** of neurons that implement the learning and an **output layer** that produces the predictions. The more layers you *stack*, the deeper the network is, hence the term deep learning.
- A **loss function**—This produces a measure of how well the network predicts the target values. Lower loss values indicate better predictions.
- An **optimizer**—This attempts to minimize the values produced by the loss function to tune the network to make better predictions.

Launch JupyterLab

This section assumes that you've activated the `tf_env` Anaconda environment you created in ection 15.3 and launched JupyterLab from the `ch15` examples folder. You can either open the `MNIST_CNN.ipynb` file in JupyterLab and execute the code in the cells we provided, or you can create a new notebook and enter the code on your own. If you prefer, you can work at the command line in IPython, however, placing your code in a Jupyter Notebook makes it significantly easier for you to *re-execute* this chapter's examples.

As a reminder, you can reset a Jupyter Notebook and remove its outputs by selecting **Restart Kernel and Clear All Outputs** from JupyterLab's **Kernel** menu. This terminates the notebook's execution and removes its outputs. You might do this if your model is not performing well and you want to try different hyperparameters or possibly restructure your neural network. [7] You can then re-execute the notebook one cell at a time or execute the entire notebook by selecting **Run All** from JupyterLab's **Run** menu.

[7] We found that we sometimes had to execute this menu option twice to clear the outputs.

15.6.1 Loading the MNIST Dataset

Let's import the **`tensorflow.keras.datasets.mnist` module** so we can load the dataset:

lick here to view code image

```
[1]: from   tensorflow.keras.datasets import mnist
```

Note that because we're using the version of Keras built into TensorFlow, the Keras module names begin with `"tensorflow."`. In the standalone Keras version, the module names begin with `"keras."`, so `keras.datasets` would be used above. Keras uses *TensorFlow* to execute the deep-learning models.

The `mnist` module's **`load_data` function** loads the MNIST training and testing sets:

lick here to view code image

```
[2]: (X_train, y_train), (X_test, y_test) =   mnist.load_data()
```

When you call `load_data` it will download the MNIST data to your system. The function returns a tuple of two elements containing the training and testing sets. Each element is itself a tuple containing the samples and labels, respectively.

15.6.2 Data Exploration

Let's get to know the data before working with it. First, we check the dimensions of the training set images (`X_train`), training set labels (`y_train`), testing set images (`X_test`) and testing set labels (`y_test`):

```
[3]: X_train.shape
[3]: (60000, 28, 28)

[4]: y_train.shape
[4]: (60000,)

[5]: X_test.shape
[5]: (10000, 28, 28)

[6]: y_test.shape
[6]: (10000,)
```

You can see from `X_train`'s and `X_test`'s shapes that the images are higher resolution than those in Scikit-learn's Digits dataset (which are 8-by-8).

Visualizing Digits

Let's visualize some of the digit images. First, enable Matplotlib in the notebook, import Matplotlib and Seaborn and set the font scale:

lick here to view code image

```
[7]: %matplotlib inline
[8]: import matplotlib.pyplot as plt

[9]: import seaborn as sns
```

C

```
[10]: sns.set(font_scale=2)
```

The IPython magic

```
%matplotlib inline
```

indicates that Matplotlib-based graphics should be displayed *in the notebook* rather than in separate windows. For more IPython magics, you can use in Jupyter Notebooks, see:

```
ttps://ipython.readthedocs.io/en/stable/interactive/magics.html
```

Next, we'll display a randomly selected set of 24 MNIST training set images. Recall from the "Array-Oriented Programming with NumPy" chapter that you can pass a sequence of indexes as a NumPy array's subscript to select only the array elements at those indexes. We'll use that capability here to select the elements at the same indexes in both the `X_train` and `y_train` arrays. This ensures that we display the correct label for each randomly selected image.

NumPy's **`choice` function** (from the `numpy.random` module) randomly selects the number of elements specified in its second argument (`24`) from the array of values in its first argument (in this case, an array containing `X_train`'s range of indices). The function returns an array containing the selected values, which we store in `index`. The expressions `X_train[index]` and `y_train[index]` use `index` to get the corresponding elements from both arrays. The rest of this cell is the visualization code from the previous chapter's Digits case study:

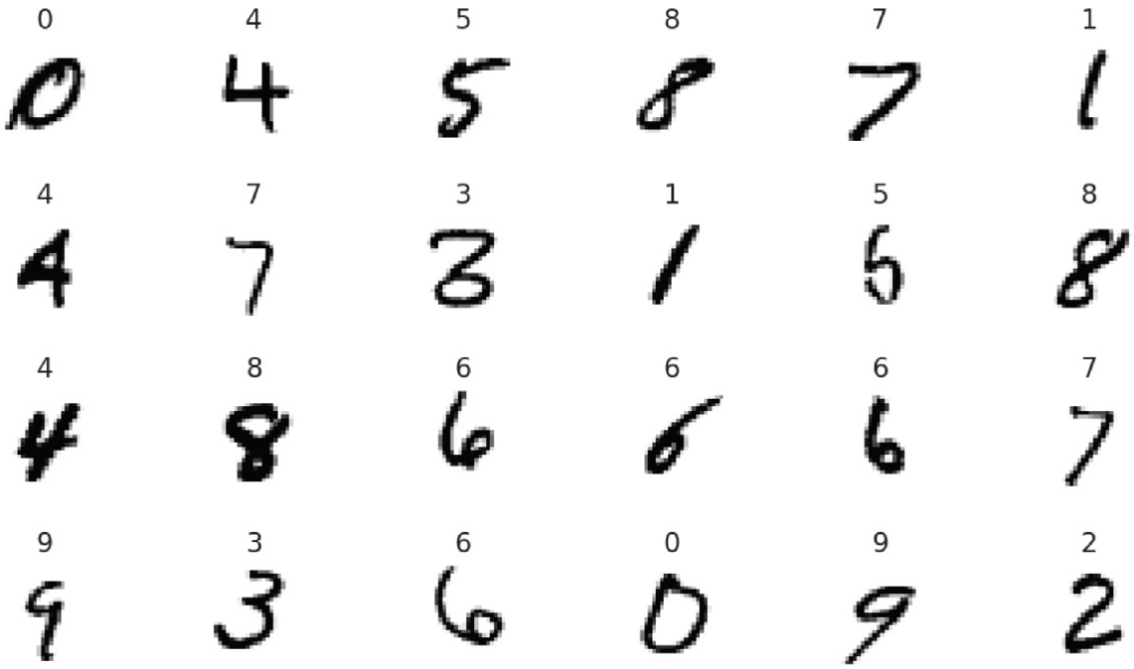
lick here to view code image

```
[11]: import numpy as np
      index =   np.random.choice(np.arange(len(X_train)), 24, replace=False)
      figure, axes =   plt.subplots(nrows=4, ncols=6, figsize=(16, 9))

      for item in zip(axes.ravel(), X_train[index], y_train[index]):
          axes, image,   target = item
            axes.imshow(image, cmap=plt.cm.gray_r)
            axes.set_xticks([])  # remove x-axis tick marks
            axes.set_yticks([])  # remove y-axis tick marks
            axes.set_title(target)
      plt.tight_layout()
```

You can see in the output below that MNIST's digit images have higher resolution than those in Scikit-learn's Digits dataset.

€



Looking at the digits, you can see why handwritten digit recognition is a challenge:

- Some people write "open" 4s (like the ones in the first and third rows), and some write "closed" 4s (like the one in the second row). Though each 4 has some similar features, they're all different from one another.
- The 3 in the second row looks strange—more like a merged 6 and 7. Compare this to the much clearer 3 in the fourth row.
- The 5 in the second row could easily be confused with a 6.
- Also, people write their digits at different angles, as you can see with the four 6s in the third and fourth rows—two are upright, one leans left and one leans right.

If you run the preceding snippet multiple times, you can see additional randomly selected digits. [8] You'll probably find that—if not for the labels displayed above each digit—it would be difficult for you to identify some of the digits. We'll soon see how accurately our first convnet will predict the digits in the MNIST test set.

[8] If you do run the cell multiple times, the snippet number next to the cell will increment each time, as it does in IPython at the command line.

15.6.3 Data Preparation

Recall from the "Machine Learning" chapter that Scikit-learn's bundled datasets were preprocessed into the shapes its models required. In real-world studies, you'll generally have to do some or all of the data preparation. The MNIST dataset requires some preparation for use in a Keras convnet.

Reshaping the Image Data

Keras convnets require NumPy array inputs in which each sample has the shape:

```
(width, height, channels)
```

For MNIST, each image's *width* and *height* are 28 pixels, and each pixel has one *channel* (the grayscale shade of the pixel from 0 to 255), so each sample's shape will be:

```
(28, 28, 1)
```

Full-color images with RGB (red/green/blue) values for each pixel, would have three *channels*—one channel each for the red, green and blue components of a color.

As the neural network learns from the images, it creates many more channels. Rather than shade or color, the learned channels will represent more complex features, like edges, curves and lines, that will eventually enable the network to recognize digits based on these additional features and how they're combined.

Let's reshape the 60,000 training and 10,000 testing set images into the correct dimensions for use in our convnet and confirm their new shapes. Recall that NumPy array method `reshape` receives a tuple representing the array's new shape:

lick here to view code image

```
[12]: X_train = X_train.reshape((60000,   28, 28, 1))

[13]: X_train.shape
[13]: (60000, 28, 28, 1)

[14]: X_test = X_test.reshape((10000,   28, 28, 1))

[15]: X_test.shape
[15]: (10000, 28, 28, 1)
```

Normalizing the Image Data

Numeric features in data samples may have value ranges that vary widely. Deep learning networks perform better on data that is scaled either into the range 0.0 to 1.0, or to a range for which the data's mean is 0.0 and its standard deviation is 1.0. [9] Getting your data into one of these forms is known as **normalization**.

[9]S. Ioffe and Szegedy, C.. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. `ttps://arxiv.org/abs/1502.03167`.

In MNIST, each pixel is an integer in the range 0–255. The following statements convert the values to 32-bit (4-byte) floating-point numbers using the NumPy array method `astype`, then divide every element in the resulting array by 255, producing normalized values in the range 0.0–1.0:

lick here to view code image

```
[16]: X_train = X_train.astype('float32')   / 255

[17]: X_test = X_test.astype('float32')   / 255
```

One-Hot Encoding: Converting the Labels From Integers to Categorical Data

As we mentioned, the convnet's prediction for each digit will be an array of 10 probabilities, indicating the likelihood that the digit belongs to a particular one of the classes 0 through 9. When we evaluate the model's accuracy, Keras compares the model's predictions to the labels. To do that, Keras requires both to have the same shape. The MNIST label for each digit, however, is one integer value in the range 0–9. So, we must transform the labels into **categorical data**—that is, arrays of categories that match the format of the predictions. To do this, we'll use a process called **one-hot encoding**, [o] which converts data into arrays of 1.0s and 0.0s in which only one element is 1.0 and the rest are 0.0s. For MNIST, the one-hot-encoded values will be 10-element arrays representing the categories 0 through 9. One-hot encoding also can be applied to other types of data.

[o] his term comes from certain digital circuits in which a group of bits is allowed to have only one bit turned on (that is, to have the value 1). ttps://en.wikipedia.org/wiki/One-ot.

We know precisely which category each digit belongs to, so the categorical representation of a digit label will consist of a 1.0 at that digit's index and 0.0s for all the other elements (again, Keras uses floating-point numbers internally). So, a 7's categorical representation is:

lick here to view code image

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0]
```

and a 3's representation is:

lick here to view code image

```
[0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

The **`tensorflow.keras.utils` module** provides function **`to_categorical`** to perform one-hot encoding. The function counts the unique categories then, for each item being encoded, creates an array of that length with a 1.0 in the correct position. Let's transform `y_train` and `y_test` from one-dimensional arrays containing the values `0`–`9` into two-dimensional arrays of categorical data. After doing so, the rows of these arrays will look like those shown above. Snippet `[21]` outputs one sample's categorical data for the digit `5` (recall that NumPy shows the decimal point, but not trailing `0`s on floating-point values):

lick here to view code image

```
 18]: from   tensorflow.keras.utils import to_categorical

[19]: y_train = to_categorical(y_train)

[20]: y_train.shape
[20]: (60000, 10)

[21]: y_train[0]
[21]: array([ 0.,  0.,  0.,  0.,  0.,  1.,  0.,  0.,  0.,    0.], dtype=float32)

[22]: y_test = to_categorical(y_test)
```

```
[23]: y_test.shape
[23]: (10000, 10)
```

5.6.4 Creating the Neural Network

Now that we've prepared the data, we'll configure a convolutional neural network. We begin with the Keras **`Sequential` model** from the **`tensorflow.keras.models` module**:

lick here to view code image

```
[24]: from   tensorflow.keras.models import Sequential

[25]: cnn = Sequential()
```

The resulting network will execute its layers sequentially—the output of one layer becomes the input to the next. This is known as a **feed-forward network**. As you'll see when we discuss *recurrent neural networks*, not all neural network operate this way.

Adding Layers to the Network

A typical convolutional neural network consists of several layers—an *input layer* that receives the training samples, *hidden layers* that learn from the samples and an *output layer* that produces the prediction probabilities. We'll create a basic convnet here. Let's import from the **`tensorflow.keras.layers` module** the layer classes we'll use in this example:

lick here to view code image

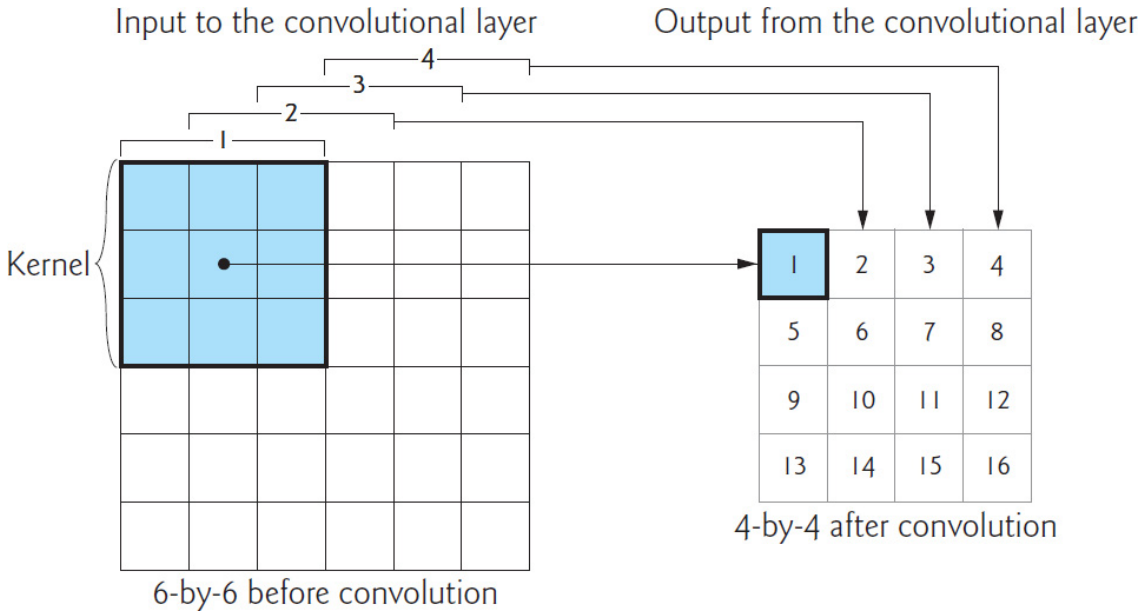
```
[26]: from   tensorflow.keras.layers import Conv2D, Dense,   Flatten,
         MaxPooling2D-
```

We discuss each below.

Convolution

We'll begin our network with a **convolution layer**, which uses the relationships between pixels that are close to one another to learn useful features (or patterns) in small areas of each sample. These features become inputs to subsequent layers.

The small areas that convolution learns from are called **kernels** or **patches**. Let's examine convolution on a 6-by-6 image. Consider the following diagram in which the 3-by-3 shaded square represents the kernel—the numbers are simply position numbers showing the order in which the kernels are visited and processed:



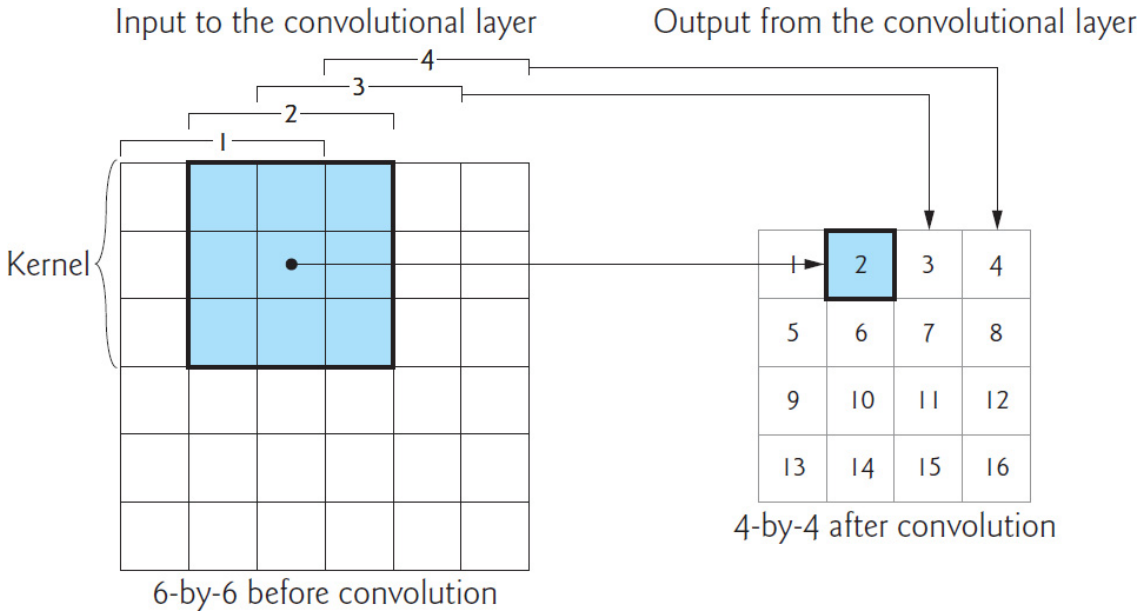
The small areas that convolution learns from are called **kernels** or **patches**. Let's examine convolution on a 6-by-6 image. Consider the following diagram in which the 3-by-3 shaded square represents the kernel—the numbers are simply position numbers showing the order in which the kernels are visited and processed:

You can think of the kernel as a "sliding window" that the convolution layer moves one pixel at a time left-to-right across the image. When the kernel reaches the right edge, the convolution layer moves the kernel one pixel down and repeats this left-to-right process. Kernels typically are 3-by-3, [1] though we found convnets that used 5-by-5 and 7-by-7 for higher-resolution images. Kernel-size is a tunable hyperparameter.

[1] `ttps://www.quora.com/How-can-I-decide-the-kernel-size-output-maps-nd-layers-of-CNN.`

Initially, the kernel is in the upper-left corner of the original image—kernel position 1 (the shaded square) in the input layer above. The convolution layer performs mathematical calculations using those *nine* features to "learn" about them, then outputs *one* new feature to position 1 in the layer's output. By looking at features near one another, the network begins to recognize features like edges, straight lines and curves.

Next, the convolution layer moves the kernel one pixel to the right (known as the *stride*) to position 2 in the input layer. This new position *overlaps* with two of the three columns in the previous position, so that the convolution layer can learn from all the features that touch one another. The layer learns from the nine features in kernel position 2 and outputs one new feature in position 2 of the output, as in:



For a 6-by-6 image and a 3-by-3 kernel, the convolution layer does this two more times to produce features for positions 3 and 4 of the layer's output. Then, the convolution layer moves the kernel one pixel down and begins the left-to-right process again for the next four kernel positions, producing outputs in positions 5–8, then 9–12 and finally 13–16. The complete pass of the image left-to-right and top-to-bottom is called a **filter**. For a 3-by-3 kernel, the filter dimensions (4-by-4 in our sample above) will be *two less than the input dimensions* (6-by-6). For each 28-by-28 MNIST image, the filter will be 26-by-26.

The number of filters in the convolutional layer is commonly 32 or 64 when processing small images like those in MNIST, and each filter produces different results. The number of filters depends on the image dimensions—higher-resolution images have more features, so they require more filters. If you study the code the Keras team used to produce their pretrained convnets, [2] you'll find that they used 64, 128 or even 256 filters in their first convolutional layers. Based on their convnets and the fact that the MNIST images are small, we'll use 64 filters in our first convolutional layer. The set of filters produced by a convolution layer is called a **feature map**.

[2] `ttps://github.com/keras-team/keras-pplications/tree/master/keras_applications.`

Subsequent convolution layers combine features from previous feature maps to recognize larger features and so on. If we were doing facial recognition, early layers might recognize lines, edges and curves, and subsequent layers might begin combining those into larger features like eyes, eyebrows, noses, ears and mouths. Once the network learns a feature, because of convolution, it can recognize that feature anywhere in the image. This is one of the reasons that convnets are used for object recognition in images.

Adding a Convolution Layer

Let's add a `Conv2D` convolution layer to our model:

lick here to view code image

```
[27]: cnn.add(Conv2D(filters=64,   kernel_size=(3, 3),   activation='relu',
                     input_shape=(28,   28, 1)))
```

he `Conv2D` layer is configured with the following arguments:

- `filters=64`—The number of filters in the resulting feature map.
- `kernel_size=(3, 3)`—The size of the kernel used in each filter.
- `activation='relu'`—The **`'relu'` (Rectified Linear Unit) activation function** is used to produce this layer's output. `'relu'` is the most widely used activation function in today's deep learning networks [3] and is good for performance because it's easy to calculate. [4] It's commonly recommended for convolutional layers. [5]

[3] Chollet, François. *Deep Learning with Python*. p. 72. Shelter Island, NY: Manning Publications, 2018.

[4] `ttps://towardsdatascience.com/exploring-activation-functions-for-eural-networks-73498da59b02.`

[5] `ttps://www.quora.com/How-should-I-choose-a-proper-activation-unction-for-the-neural-network.`

Because this is the first layer in the model, we also pass the `input_shape=(28, 28,1)` argument to specify the shape of each sample. This automatically creates an input layer to load the samples and pass them into the `Conv2D` layer, which is actually the first *hidden* layer. In Keras, each subsequent layer infers its `input_shape` from the previous layer's output shape, making it easy to *stack* layers.

Dimensionality of the First Convolution Layer's Output

In the preceding convolutional layer, the input samples are 28-by-28-by-1—that is, 784 features each. We specified 64 filters and a 3-by-3 kernel size for the layer, so the output for each image is 26-by-26-by-64 for a total of 43,264 features in the feature map—a significant increase in dimensionality and an enormous number compared to the numbers of features we processed in the "Machine Learning" chapter's models. As each layer adds more features, the resulting feature maps' *dimensionality* becomes significantly larger. This is one of the reasons that deep learning studies often require tremendous processing power.

Overfitting

Recall from the previous chapter, that overfitting can occur when your model is too complex compared to what it is modeling. In the most extreme case, a model memorizes its training data. When you make predictions with an overfit model, they will be accurate if new data matches the training data, but the model could perform poorly with data it has never seen.

Overfitting tends to occur in deep learning as the dimensionality of the layers becomes too large. [6, 7, 8] This causes the network to learn *specific* features of the training-set digit images, rather than learning the *general* features of digit images. Some techniques to prevent overfitting include training for fewer epochs, data augmentation, dropout and L1 or L2 regularization. [9, 0] We'll discuss dropout later in the chapter.

[6] `ttps://cs231n.github.io/convolutional-networks/.`

[7] ttps://medium.com/@cxu24/why-dimensionality-reduction-is-mportant-dd60b5611543.

[8] ttps://towardsdatascience.com/preventing-deep-neural-network-from-verfitting-953458db800a.

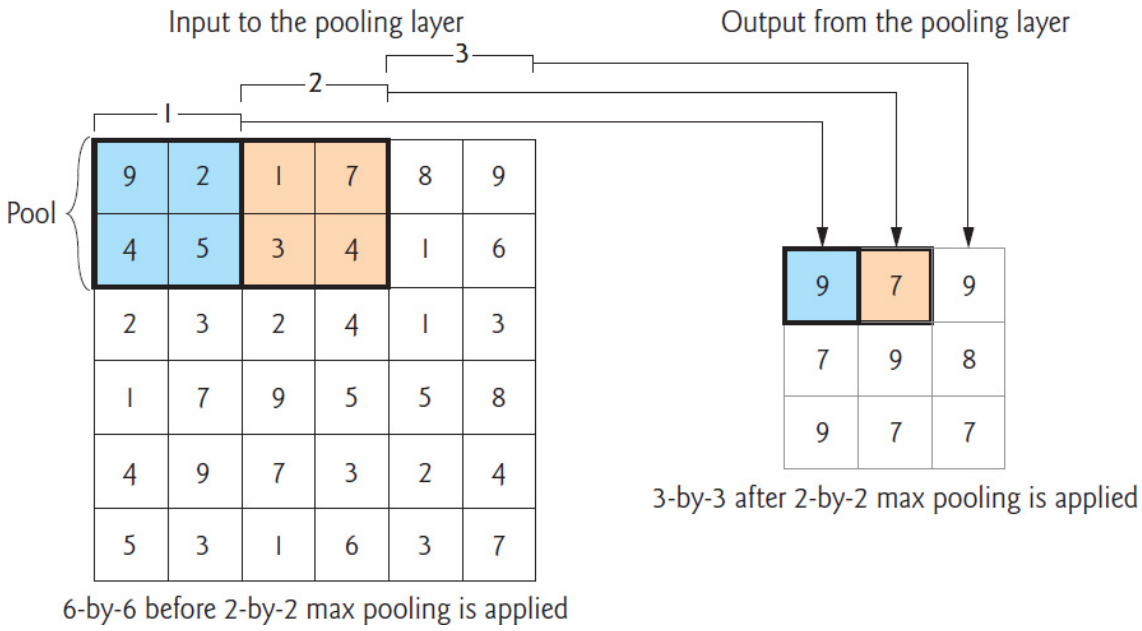
[9] ttps://towardsdatascience.com/deep-learning-3-more-on-cnns-andling-overfitting-2bd5d99abe5d.

[0] ttps://www.kdnuggets.com/2015/04/preventing-overfitting-neural-etworks.html.

igher dimensionality also increases (and sometimes explodes) computation time. If you're performing the deep learning on CPUs rather than GPUs or TPUs, the training could become intolerably slow.

Adding a Pooling Layer

To reduce overfitting and computation time, a convolution layer is often followed by one or more layers that *reduce the dimensionality* of the convolution layer's output. A **pooling layer** *compresses* (or *down-samples*) the results by discarding features, which helps make the model more general. The most common pooling technique is called **max pooling**, which examines a 2-by-2 square of features and keeps only the maximum feature. To understand pooling, let's once again assume a 6-by-6 set of features. In the following diagram, the numeric values in the 6-by-6 square represent the features that we wish to compress and the 2-by-2 blue square in position 1 represents the initial pool of features to examine:



The max pooling layer first looks at the pool in position 1 above, then outputs the *maximum* feature from that pool—9 in our diagram. Unlike convolution, there's *no overlap* between pools. The pool moves by its width—for a 2-by-2 pool, the *stride* is 2. For the second pool, represented by the orange 2-by-2 square, the layer outputs 7. For the third pool, the layer outputs 9. Once the pool reaches the right edge, the pooling layer moves the pool down by its height—2 rows—then continues from left-to-right. Because every group of four features is reduced to one, 2-by-2 pooling *compresses* the number of features by 75%.

Let's add a **MaxPooling2D** layer to our model:

lick here to view code image

```
[28]: cnn.add(MaxPooling2D(pool_size=(2, 2)))
```

This reduces the previous layer's output from 26-by-26-by-64 to 13-by-13-by-64. [1]

[1]Another technique for reducing overfitting is to add `Dropout` layers.

Though pooling is a common technique to reduce overfitting, some research suggests that additional convolutional layers which use larger strides for their kernels can reduce dimensionality and overfitting *without* discarding features. [2]

[2]Tobias, Jost, Dosovitskiy, Alexey, Brox, Thomas, Riedmiller, and Martin. Striving for Simplicity: The All Convolutional Net. April 13, 2015. `ttps://arxiv.org/abs/1412.6806`.

Adding Another Convolutional Layer and Pooling Layer

Convnets often have many convolution and pooling layers. The Keras team's convnets tend to double the number of filters in subsequent convolutional layers to enable the model to learn more relationships between the features. [3] So, let's add a second convolution layer with 128 filters, followed by a second pooling layer to once again reduce the dimensionality by 75%:

[3] `ttps://github.com/keras-team/keras-pplications/tree/master/keras_applications`.

lick here to view code image

```
[29]: cnn.add(Conv2D(filters=128,   kernel_size=(3, 3),   activation='relu'))

[30]: cnn.add(MaxPooling2D(pool_size=(2, 2)))
```

The input to the second convolution layer is the 13-by-13-by-64 output of the first pooling layer. So, the output of snippet `[29]` will be 11-by-11-by-128. For odd dimensions like 11-by-11, Keras pooling layers round *down* by default (in this case to 10-by-10), so this pooling layer's output will be 5-by-5-by-128.

Flattening the Results

At this point, the previous layer's output is three-dimensional (5-by-5-by-128), but the final output of our model will be a *one-dimensional* array of 10 probabilities that classify the digits. To prepare for the one-dimensional final predictions, we first need to *flatten* the previous layer's three-dimensional output. A Keras **Flatten** layer reshapes its input to one dimension. In this case, the `Flatten` layer's output will be 1-by-3200 (that is, `5 * 5 * 128`):

```
[31]: cnn.add(Flatten())
```

Adding a Dense Layer to Reduce the Number of Features

The layers before the `Flatten` layer learned digit features. Now we need to take all those features and learn the relationships among them so our model can classify which digit each image represents. Learning the relationships among features and performing classification is accomplished with fully connected **`Dense`** layers, like those shown in the neural network diagram earlier in the chapter. The following `Dense` layer creates 128 neurons (`units`) that learn from the 3200 outputs of the previous layer:

lick here to view code image

```
[32]: cnn.add(Dense(units=128,   activation='relu'))
```

Many convnets contain at least one `Dense` layer like the one above. Convnets geared to more complex image datasets with higher-resolution images like Image-Net—a dataset of over 14 million images [4]—often have several `Dense` layers, commonly with 4096 neurons. You can see such configurations in several of Keras's pretrained Image-Net convnets [5]—we list these in ection 15.11.

[4] ttp://www.image-net.org.

[5] ttps://github.com/keras-team/keras-pplications/tree/master/keras_applications.

Adding Another Dense Layer to Produce the Final Output

Our final layer is a `Dense` layer that classifies the inputs into neurons representing the classes 0 through 9. The **`softmax` activation function** converts the values of these remaining 10 neurons into classification probabilities. The neuron that produces the highest probability represents the prediction for a given digit image:

lick here to view code image

```
[33]: cnn.add(Dense(units=10,   activation='softmax'))
```

Printing the Model's Summary

A model's **`summary` method** shows you the model's layers. Some interesting things to note are the output shapes of the various layers and the number of parameters. The parameters are the *weights* that the network learns during training. [6, 7] This is a relatively small network, yet it will need to learn nearly 500,000 parameters! And this is for tiny images that have less than one quarter of the resolution of the icons on most smartphone home screens. Imagine how many features a network would have to learn to process high-resolution 4K video frames or the super-high-resolution images produced by today's digital cameras. In the `Output Shape`, `None` simply means that the model does not know in advance how many training samples you're going to provide—this is known only when you start the training.

[6] ttps://hackernoon.com/everything-you-need-to-know-about-neural-

etworks-8988c3ee4491.

[7] ttps://www.kdnuggets.com/2018/06/deep-learning-best-practices-eight-initialization-.html.

lick here to view code image

```
[34]: cnn.summary()
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 26, 26, 64)        640
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 13, 13, 64)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 11, 11, 128)       73856
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 5, 5, 128)         0
_________________________________________________________________
flatten_1 (Flatten)          (None, 3200)              0
_________________________________________________________________
dense_1 (Dense)              (None, 128)               409728
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1290
=================================================================
Total params: 485,514
Trainable params: 485,514
Non-trainable params: 0
_________________________________________________________________
```

Also, note that there are no "non-trainable" parameters. By default, Keras trains *all* parameters, but it is possible to prevent training for specific layers, which is typically done when you're tuning your networks or using another model's learned parameters in a new model (a process called *transfer learning*). [8]

[8] ttps://keras.io/getting-started/faq/#how-can-i-freeze-keras-ayers.

Visualizing a Model's Structure

You can visualize the model summary using the **`plot_model` function** from the module `tensorflow.keras.utils`:

lick here to view code image

```
[35]: from   tensorflow.keras.utils import plot_model
      from IPython.display import Image
      plot_model(cnn, to_file='convnet.png',   show_shapes=True,
                 show_layer_names=True)
      Image(filename='convnet.png')
```

After storing the visualization in `convnet.png`, we use module `IPython.display`'s **`Image` class** to show the image in the notebook. Keras assigns the layer names in the image: [9]

[9]The node with the large integer value 112430057960 at the top of the diagram appears to be